

Practical 5

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# 1. Load the dataset
df = pd.read_csv('Social_Network_Ads(5).csv')

# 2. We use 'Age' and 'EstimatedSalary' as independent variables (X)
# We use 'Purchased' as the dependent variable (y)
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']

# 3. Split into 80% Training and 20% Testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)

print("Data Loaded and Split successfully!")

# Logistic regression works best when features are on the same scale
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

print("Features Scaled (Age and Salary are now in a similar range).")
```

```
from sklearn.linear_model import LogisticRegression
```

```
# 1. Initialize the model
```

```
model = LogisticRegression()
```

```
# 2. Train the model
```

```
model.fit(X_train, y_train)
```

```
# 3. Make predictions on the test set
```

```
y_pred = model.predict(X_test)
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
```

```
# 1. Compute Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
TN, FP, FN, TP = cm.ravel()
```

```
# 2. Calculate required metrics
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

```
error_rate = 1 - accuracy
```

```
# 3. Print everything
```

```
print(f"Confusion Matrix:\n{cm}")
```

```
print(f"\nTrue Positives (TP): {TP}")
```

```
print(f"False Positives (FP): {FP}")
```

```
print(f"True Negatives (TN): {TN}")
```

```
print(f"False Negatives (FN): {FN}")
```

```
print(f"\nAccuracy: {accuracy:.2f}")  
print(f"Error Rate: {error_rate:.2f}")  
print(f"Precision: {precision:.2f}")  
print(f"Recall: {recall:.2f}")
```